KCA UNIVERSITY

FACULTY OF COMPUTING AND INFORMATION MANAGEMENT
DEPARTMENT OF SOFTWARE DEVELOPMENT & INFORMATION SYSTEMS

KCAU SMART DINING SYSTEM

Continuous Assessment Test 1 (CAT 1) — Comprehensive Java Backend Source Code Document

OFFICIAL DELIVERABLE FILE: KCAU_SMART_DINING_JAVA_SOURCE_CODE.pdf

Student Name / Author	Solomon (solomon476)
Project Title	KCAU Smart Dining System (Campus Cafeteria Automation)
Academic Course / Unit	Continuous Assessment Test 1 (CAT 1)
Document Specification	Java Source Code Document (Item 2 of Assessment Portfolio)
Backend Architecture	Modular Java 21/25, Project Loom Virtual Threads, SQLite JDBC
Source Code Scope	23 Java Classes (19 Modular Engine Classes + 4 Compatibility Aliases)
Live Web Deployment	https://kca-cafeteria.vercel.app
GitHub Repository	https://github.com/solomon476/KCAU-SMART-DINING-SYSTEM

Note on Document Formatting: This document compiles the complete source code for the KCAU Smart Dining System backend. Per CAT 1 instructions, all code is formatted using Consolas monospaced typography with preserved indentation, complete imports, package declarations, and each class starting on a new page.

Backend Architecture & Source Code Directory

The backend is structured into clear separation-of-concerns layers following modern enterprise Java best practices. The table below indexes all 23 Java source files included in this document:

#	Class Name	Package / Directory	Architectural Tier	Lines
1	**Server.java**	com.kcau.dining	Core Server & REST Dispatcher	189
2	**User.java**	com.kcau.dining.model	Domain Model	66
3	**MenuItem.java**	com.kcau.dining.model	Domain Model	64
4	**Order.java**	com.kcau.dining.model	Domain Model	72
5	**OrderItem.java**	com.kcau.dining.model	Domain Model	61
6	**Payment.java**	com.kcau.dining.model	Domain Model	64
7	**DatabaseConnection.java**	com.kcau.dining.database	Persistence Infrastructure	40
8	**DatabaseInitializer.java**	com.kcau.dining.database	Persistence Infrastructure	210
9	**UserService.java**	com.kcau.dining.service	Business Logic Service	122
10	**MenuService.java**	com.kcau.dining.service	Business Logic Service	125
11	**OrderService.java**	com.kcau.dining.service	Business Logic Service	190
12	**PaymentService.java**	com.kcau.dining.service	Business Logic Service	88
13	**ReportService.java**	com.kcau.dining.service	Business Logic Service	83
14	**HttpHelper.java**	com.kcau.dining.handler	REST & Security Infrastructure	184
15	**UserHandler.java**	com.kcau.dining.handler	REST API Controller	146
16	**MenuHandler.java**	com.kcau.dining.handler	REST API Controller	143
17	**OrderHandler.java**	com.kcau.dining.handler	REST API Controller	207
18	**PaymentHandler.java**	com.kcau.dining.handler	REST API Controller	90
19	**ReportHandler.java**	com.kcau.dining.handler	REST API Controller	39
20	**Server.java (Root Entrypoint)**	(default package)	Root Application Launcher	26
21	**User.java (Root Model Alias)**	(default package)	Architectural Compatibility Alias	13
22	**MenuItem.java (Root Model Alias)**	(default package)	Architectural Compatibility Alias	13
23	**Order.java (Root Model Alias)**	(default package)	Architectural Compatibility Alias	13

Class 1: Server.java

Package: com.kcau.dining
Source Path: src/main/java/com/kcau/dining/Server.java
Layer: Core Server & REST Dispatcher
Total Lines: 189 lines of Java source code

Functional Role: Central Virtual-Thread HTTP server entrypoint. Initializes SQLite database schema, binds HTTP listener (port 3000), registers REST API context handlers, serves static web assets with SPA fallback, and manages CORS headers.

Complete Java Source Code:

```java
package com.kcau.dining;

import com.kcau.dining.database.DatabaseConnection;
import com.kcau.dining.database.DatabaseInitializer;
import com.kcau.dining.handler.*;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.File;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.net.URLConnection;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.Instant;
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.Executors;

/**
 * KCAU Smart Dining System - Main Backend Server Entrypoint
 * Standalone virtual-threaded Java HTTP server coordinating SQLite database and REST endpoints.
 */
public class Server {

    private static final int DEFAULT_PORT = 3000;
    private static final Path PUBLIC_DIR = Paths.get("public").toAbsolutePath();

    public static void main(String[] args) throws IOException {
        // 1. Initialize SQLite Database Schema & Seed Data
        DatabaseInitializer.init();

        // 2. Determine Port
        int port = DEFAULT_PORT;
        String envPort = System.getenv("PORT");
        if (envPort != null && !envPort.isBlank()) {
            try {
                port = Integer.parseInt(envPort.trim());
            } catch (NumberFormatException ignored) {}
        } else if (args != null && args.length > 0) {
            try {
                port = Integer.parseInt(args[0].trim());
            } catch (NumberFormatException ignored) {}
        }

        HttpServer server;
        try {
            server = HttpServer.create(new InetSocketAddress(port), 0);
        } catch (IOException e) {
```

```
            int fallbackPort = (port == 3000) ? 8080 : 3001;
            System.out.printf("⚠️ Port %d unavailable, switching to port %d...%n", port, fallbackPort);
            port = fallbackPort;
            server = HttpServer.create(new InetSocketAddress(port), 0);
        }

        // 3. Register Modular Handlers
        UserHandler userHandler = new UserHandler();
        MenuHandler menuHandler = new MenuHandler();
        OrderHandler orderHandler = new OrderHandler();
        PaymentHandler paymentHandler = new PaymentHandler();
        ReportHandler reportHandler = new ReportHandler();

        // Auth & User routes
        server.createContext("/api/signup", userHandler);
        server.createContext("/api/login", userHandler);
        server.createContext("/api/me", userHandler);
        server.createContext("/api/users", userHandler);

        // Menu routes
        server.createContext("/api/menu", menuHandler);

        // Order routes
        server.createContext("/api/orders", orderHandler);

        // Payment routes
        server.createContext("/api/payments", paymentHandler);

        // Report & Analytics routes
        server.createContext("/api/reports", reportHandler);

        // System Health & Diagnostics route
        server.createContext("/api/status", new StatusHandler());

        // Static files and Single Page App (SPA) fallback
        server.createContext("/", new StaticFileHandler());

        // 4. Configure Virtual Thread Executor (Project Loom)
        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        server.start();

        System.out.println("================================================");
        System.out.println("☕ KCAU Smart Dining System - Modular Java Backend");
        System.out.printf("🚀 Server running on http://localhost:%d%n", port);
        System.out.println("📁 Serving static files from: " + PUBLIC_DIR);
        System.out.println("💾 Database: SQLite (" + DatabaseConnection.getDbPath() + ")");
        System.out.println("🔐 Security: PBKDF2 Hashing + HMAC-SHA256 JWT");
        System.out.println("📋 Roles Enabled: Student, Dining Staff, Admin");
        System.out.println("================================================");
    }

    // ------------------------------------------------------------
    // Status Handler
    // ------------------------------------------------------------
    static class StatusHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            if (HttpHelper.handleCorsPreflight(exchange)) return;

            Map<String, Object> status = new HashMap<>();
            status.put("status", "KCAU Smart Dining Java Backend is running!");
            status.put("database", "SQLite (" + DatabaseConnection.getDbPath() + ")");
            status.put("sqliteExists", new File(DatabaseConnection.getDbPath()).exists());
```

```
        status.put("timestamp", Instant.now().toString());
        status.put("runtime", System.getProperty("java.version"));
        HttpHelper.sendJson(exchange, 200, status);
    }
}

// ------------------------------------------------------------
// Static Files & SPA Fallback Handler
// ------------------------------------------------------------
static class StaticFileHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange exchange) throws IOException {
        if (HttpHelper.handleCorsPreflight(exchange)) return;

        String rawPath = exchange.getRequestURI().getPath();
        String pathStr = URLDecoder.decode(rawPath, StandardCharsets.UTF_8);

        if (pathStr.startsWith("/api/")) {
            HttpHelper.sendError(exchange, 404, "API endpoint not found");
            return;
        }

        Path requestedFile;
        if ("/".equals(pathStr) || pathStr.isBlank()) {
            requestedFile = PUBLIC_DIR.resolve("index.html").normalize();
        } else {
            String subPath = pathStr.startsWith("/") ? pathStr.substring(1) : pathStr;
            requestedFile = PUBLIC_DIR.resolve(subPath).normalize();
        }

        // Prevent path traversal
        if (!requestedFile.startsWith(PUBLIC_DIR)) {
            HttpHelper.sendError(exchange, 403, "Access Denied");
            return;
        }

        // SPA Fallback: if not found or is directory, serve index.html
        if (!Files.exists(requestedFile) || Files.isDirectory(requestedFile)) {
            requestedFile = PUBLIC_DIR.resolve("index.html").normalize();
        }

        if (!Files.exists(requestedFile)) {
            HttpHelper.sendError(exchange, 404, "Frontend file not found");
            return;
        }

        String contentType = determineContentType(requestedFile);
        exchange.getResponseHeaders().set("Content-Type", contentType);

        byte[] fileBytes = Files.readAllBytes(requestedFile);
        exchange.sendResponseHeaders(200, fileBytes.length);
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(fileBytes);
        }
    }

    private String determineContentType(Path file) {
        String fileName = file.getFileName().toString().toLowerCase();
        if (fileName.endsWith(".html")) return "text/html; charset=utf-8";
        if (fileName.endsWith(".css")) return "text/css; charset=utf-8";
        if (fileName.endsWith(".js")) return "application/javascript; charset=utf-8";
        if (fileName.endsWith(".json")) return "application/json; charset=utf-8";
        if (fileName.endsWith(".png")) return "image/png";
```

```
            if (fileName.endsWith(".jpg") || fileName.endsWith(".jpeg")) return "image/jpeg";
            if (fileName.endsWith(".svg")) return "image/svg+xml";
            if (fileName.endsWith(".ico")) return "image/x-icon";
            if (fileName.endsWith(".webp")) return "image/webp";
            return URLConnection.guessContentTypeFromName(fileName);
        }
    }
}
```

Class 2: User.java

Package: com.kcau.dining.model
Source Path: src/main/java/com/kcau/dining/model/User.java
Layer: Domain Model
Total Lines: 66 lines of Java source code

Functional Role: Encapsulates user profile and authentication identity. Stores unique user ID, full name, student ID / username, PBKDF2 password hash, and authorization role ('student', 'staff', or 'admin').

Complete Java Source Code:

```java
package com.kcau.dining.model;

import java.time.Instant;

/**
 * Represents a student, staff, or administrator user in the Smart Dining System.
 */
public class User {
    private String id;
    private String name;
    private String studentId;
    private String passwordHash;
    private String role; // "student", "staff", "admin"
    private Instant createdAt;

    public User() {
        this.role = "student";
        this.createdAt = Instant.now();
    }

    public User(String id, String name, String studentId, String passwordHash) {
        this(id, name, studentId, passwordHash, "student", Instant.now());
    }

    public User(String id, String name, String studentId, String passwordHash, String role) {
        this(id, name, studentId, passwordHash, role, Instant.now());
    }

    public User(String id, String name, String studentId, String passwordHash, String role, Instant createdAt) {
        this.id = id;
        this.name = name;
        this.studentId = studentId;
        this.passwordHash = passwordHash;
        this.role = (role != null && !role.isBlank()) ? role.toLowerCase() : "student";
        this.createdAt = (createdAt != null) ? createdAt : Instant.now();
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getStudentId() { return studentId; }
    public void setStudentId(String studentId) { this.studentId = studentId; }

    public String getPasswordHash() { return passwordHash; }
    public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }

    public String getRole() { return role; }
    public void setRole(String role) { this.role = (role != null) ? role.toLowerCase() : "student"; }

    public Instant getCreatedAt() { return createdAt; }
    public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
```

```
    @Override
    public String toString() {
        return "User{" +
                "id='" + id + '\'' +
                ", name='" + name + '\'' +
                ", studentId='" + studentId + '\'' +
                ", role='" + role + '\'' +
                ", createdAt=" + createdAt +
                '}';
    }
}
```

Class 3: MenuItem.java

Package: com.kcau.dining.model	**Layer:** Domain Model
Source Path: src/main/java/com/kcau/dining/model/MenuItem.java	**Total Lines:** 64 lines of Java source code

Functional Role: Represents cafeteria food and drink catalog items. Stores item ID, name, description, price (KES), category classification ('breakfast', 'lunch', 'drinks', 'fruits', 'snacks'), image URL, and stock availability flag.

Complete Java Source Code:

```java
package com.kcau.dining.model;

/**
 * Represents a cafeteria food/drink menu item.
 */
public class MenuItem {
    private String id;
    private String name;
    private String description;
    private double price;    // in KES
    private String category; // "breakfast", "lunch", "drinks", "fruits", "snacks"
    private String image;
    private boolean available;

    public MenuItem() {
        this.available = true;
    }

    public MenuItem(String id, String name, String category, double price, String image) {
        this(id, name, "", category, price, image, true);
    }

    public MenuItem(String id, String name, String description, String category, double price, String image, boolean available) {
        this.id = id;
        this.name = name;
        this.description = description != null ? description : "";
        this.category = category;
        this.price = price;
        this.image = image;
        this.available = available;
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    public String getImage() { return image; }
    public void setImage(String image) { this.image = image; }

    public boolean isAvailable() { return available; }
    public void setAvailable(boolean available) { this.available = available; }

    @Override
    public String toString() {
```

```
        return "MenuItem{" +
                "id='" + id + '\'' +
                ", name='" + name + '\'' +
                ", category='" + category + '\'' +
                ", price=" + price +
                ", available=" + available +
                '}';
    }
}
```

Class 4: Order.java

Package: com.kcau.dining.model
Source Path: src/main/java/com/kcau/dining/model/Order.java
Layer: Domain Model
Total Lines: 72 lines of Java source code

Functional Role: Customer order header entity. Contains order UUID, student/customer ID, customer name, total amount (KES), lifecycle state ('PENDING', 'PREPARING', 'READY', 'COLLECTED', 'CANCELLED'), creation timestamp, and child OrderItem list.

Complete Java Source Code:

```java
package com.kcau.dining.model;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;

/**
 * Represents an order placed by a student or staff member.
 */
public class Order {
    private String orderId;
    private String userId;
    private String customerName;
    private double totalAmount;
    private String status; // "PENDING", "PREPARING", "READY", "COLLECTED", "CANCELLED"
    private Instant createdAt;
    private List<OrderItem> items = new ArrayList<>();

    public Order() {
        this.status = "PENDING";
        this.createdAt = Instant.now();
    }

    public Order(String orderId, String userId, String customerName, double totalAmount, String status, Instant createdAt) {
        this.orderId = orderId;
        this.userId = userId;
        this.customerName = customerName;
        this.totalAmount = totalAmount;
        this.status = (status != null && !status.isBlank()) ? status : "PENDING";
        this.createdAt = (createdAt != null) ? createdAt : Instant.now();
    }

    public String getOrderId() { return orderId; }
    public void setOrderId(String orderId) { this.orderId = orderId; }

    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }

    // Backward compatibility helper for studentId
    public String getStudentId() { return userId; }
    public void setStudentId(String studentId) { this.userId = studentId; }

    public String getCustomerName() { return customerName; }
    public void setCustomerName(String customerName) { this.customerName = customerName; }

    public double getTotalAmount() { return totalAmount; }
    public void setTotalAmount(double totalAmount) { this.totalAmount = totalAmount; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public Instant getCreatedAt() { return createdAt; }
    public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }

    public List<OrderItem> getItems() { return items; }
```

```
    public void setItems(List<OrderItem> items) { this.items = items; }

    public void addItem(OrderItem item) {
        this.items.add(item);
    }

    @Override
    public String toString() {
        return "Order{" +
                "orderId='" + orderId + '\'' +
                ", customerName='" + customerName + '\'' +
                ", totalAmount=" + totalAmount +
                ", status='" + status + '\'' +
                ", itemsCount=" + items.size() +
                '}';
    }
}
```

Class 5: OrderItem.java

Package: com.kcau.dining.model	**Layer:** Domain Model
Source Path: src/main/java/com/kcau/dining/model/OrderItem.java	**Total Lines:** 61 lines of Java source code

Functional Role: Represents a distinct line item within a customer order. Tracks internal row ID, foreign key reference to order ID, food item ID, food item name, unit price (KES), quantity ordered, and calculates line subtotal.

Complete Java Source Code:

```java
package com.kcau.dining.model;

/**
 * Represents a single line item within a cafeteria customer order.
 */
public class OrderItem {
    private int id;
    private String orderId;
    private String foodId;
    private String foodName;
    private int quantity;
    private double unitPrice;

    public OrderItem() {}

    public OrderItem(String foodId, String foodName, int quantity, double unitPrice) {
        this.foodId = foodId;
        this.foodName = foodName;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
    }

    public OrderItem(int id, String orderId, String foodId, String foodName, int quantity, double unitPrice) {
        this.id = id;
        this.orderId = orderId;
        this.foodId = foodId;
        this.foodName = foodName;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getOrderId() { return orderId; }
    public void setOrderId(String orderId) { this.orderId = orderId; }

    public String getFoodId() { return foodId; }
    public void setFoodId(String foodId) { this.foodId = foodId; }

    public String getFoodName() { return foodName; }
    public void setFoodName(String foodName) { this.foodName = foodName; }

    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }

    public double getUnitPrice() { return unitPrice; }
    public void setUnitPrice(double unitPrice) { this.unitPrice = unitPrice; }

    public double getSubtotal() { return quantity * unitPrice; }

    @Override
    public String toString() {
        return "OrderItem{" +
                "foodName='" + foodName + '\'' +
```

```
                ", quantity=" + quantity +
                ", unitPrice=" + unitPrice +
                ", subtotal=" + getSubtotal() +
                '}';
    }
}
```

Class 6: Payment.java

Package: com.kcau.dining.model	**Layer:** Domain Model
Source Path: src/main/java/com/kcau/dining/model/Payment.java	**Total Lines:** 64 lines of Java source code

Functional Role: Represents a financial transaction record in the cafeteria ledger. Tracks payment ID, order ID reference, payment method ('MPESA', 'CASH', 'CARD'), amount (KES), transaction reference code, status, and completion timestamp.

Complete Java Source Code:

```java
package com.kcau.dining.model;

import java.time.Instant;

/**
 * Represents a payment transaction (simulated or real) for a dining order.
 */
public class Payment {
    private String id;
    private String orderId;
    private String method;    // "MPESA", "AIRTEL", "CARD", "CASH"
    private double amount;
    private String status;    // "COMPLETED", "SIMULATED", "PENDING", "FAILED"
    private String reference; // Transaction reference code e.g. QA89X45Y7Z
    private Instant createdAt;

    public Payment() {
        this.status = "SIMULATED";
        this.createdAt = Instant.now();
    }

    public Payment(String id, String orderId, String method, double amount, String status, String reference, Instant createdAt) {
        this.id = id;
        this.orderId = orderId;
        this.method = method;
        this.amount = amount;
        this.status = (status != null && !status.isBlank()) ? status : "SIMULATED";
        this.reference = reference;
        this.createdAt = (createdAt != null) ? createdAt : Instant.now();
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getOrderId() { return orderId; }
    public void setOrderId(String orderId) { this.orderId = orderId; }

    public String getMethod() { return method; }
    public void setMethod(String method) { this.method = method; }

    public double getAmount() { return amount; }
    public void setAmount(double amount) { this.amount = amount; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public String getReference() { return reference; }
    public void setReference(String reference) { this.reference = reference; }

    public Instant getCreatedAt() { return createdAt; }
    public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }

    @Override
    public String toString() {
        return "Payment{" +
```

```
            "id='" + id + '\'' +
            ", orderId='" + orderId + '\'' +
            ", method='" + method + '\'' +
            ", amount=" + amount +
            ", status='" + status + '\'' +
            ", reference='" + reference + '\'' +
            '}';
    }
}
```

Class 7: DatabaseConnection.java

Package: com.kcau.dining.database
Source Path: src/main/java/com/kcau/dining/database/DatabaseConnection.java
Layer: Persistence Infrastructure
Total Lines: 40 lines of Java source code

Functional Role: Provides thread-safe JDBC connection pooling to the local SQLite database ('data/dining.db'). Ensures automatic directory creation and reliable connection retrieval across parallel virtual threads.

Complete Java Source Code:

```java
package com.kcau.dining.database;

import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * Manages SQLite database connectivity for the KCAU Smart Dining System.
 */
public class DatabaseConnection {

    private static final String DB_PATH = "data/dining.db";
    private static final String JDBC_URL = "jdbc:sqlite:" + DB_PATH;

    static {
        try {
            Class.forName("org.sqlite.JDBC");
        } catch (ClassNotFoundException e) {
            System.err.println("⚠️ SQLite JDBC Driver not found in classpath. Ensure lib/sqlite-jdbc.jar is included.");
        }
    }

    /**
     * Gets a new connection to the SQLite database.
     * Ensures parent directories exist.
     */
    public static Connection getConnection() throws SQLException {
        File dbFile = new File(DB_PATH);
        File parentDir = dbFile.getParentFile();
        if (parentDir != null && !parentDir.exists()) {
            parentDir.mkdirs();
        }
        return DriverManager.getConnection(JDBC_URL);
    }

    public static String getDbPath() {
        return DB_PATH;
    }
}
```

Class 8: DatabaseInitializer.java

Package: com.kcau.dining.database
Source Path: src/main/java/com/kcau/dining/database/DatabaseInitializer.java
Layer: Persistence Infrastructure
Total Lines: 210 lines of Java source code

Functional Role: Executes automated DDL table definitions with foreign keys for users, menu_items, orders, order_items, and payments. Seeds initial accounts (students, kitchen staff, admin) and 12+ cafeteria menu items.

Complete Java Source Code:

```
package com.kcau.dining.database;

import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.time.Instant;
import java.util.Base64;
import java.util.UUID;

/**
 * Initializes the SQLite database schema and seeds initial cafeteria data.
 */
public class DatabaseInitializer {

    public static void init() {
        try (Connection conn = DatabaseConnection.getConnection();
             Statement stmt = conn.createStatement()) {

            // 1. Users table (Student ID, Name, Password Hash, Role)
            stmt.execute("""
                CREATE TABLE IF NOT EXISTS users (
                    id TEXT PRIMARY KEY,
                    name TEXT NOT NULL,
                    student_id TEXT UNIQUE NOT NULL,
                    password_hash TEXT NOT NULL,
                    role TEXT NOT NULL DEFAULT 'student',
                    created_at TEXT NOT NULL
                );
            """);

            // 2. Menu Items table (Food ID, Name, Description, Price, Availability, Category, Image)
            stmt.execute("""
                CREATE TABLE IF NOT EXISTS menu_items (
                    id TEXT PRIMARY KEY,
                    name TEXT NOT NULL,
                    description TEXT,
                    price REAL NOT NULL,
                    category TEXT NOT NULL,
                    image TEXT,
                    available INTEGER NOT NULL DEFAULT 1
                );
            """);

            // 3. Orders table (Order ID, User ID, Customer Name, Date, Total, Order Status)
            stmt.execute("""
                CREATE TABLE IF NOT EXISTS orders (
                    id TEXT PRIMARY KEY,
                    user_id TEXT NOT NULL,
                    customer_name TEXT NOT NULL,
                    date TEXT NOT NULL,
```

```
                total REAL NOT NULL,
                order_status TEXT NOT NULL DEFAULT 'PENDING'
            );
        """);

        // 4. Order Items table (Order ID, Food ID, Food Name, Quantity, Unit Price)
        stmt.execute("""
            CREATE TABLE IF NOT EXISTS order_items (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                order_id TEXT NOT NULL,
                food_id TEXT NOT NULL,
                food_name TEXT NOT NULL,
                quantity INTEGER NOT NULL,
                unit_price REAL NOT NULL,
                FOREIGN KEY(order_id) REFERENCES orders(id) ON DELETE CASCADE
            );
        """);

        // 5. Payments table (Payment ID, Order ID, Method, Amount, Status, Reference, Date)
        stmt.execute("""
            CREATE TABLE IF NOT EXISTS payments (
                id TEXT PRIMARY KEY,
                order_id TEXT NOT NULL,
                method TEXT NOT NULL,
                amount REAL NOT NULL,
                status TEXT NOT NULL,
                reference TEXT NOT NULL,
                created_at TEXT NOT NULL,
                FOREIGN KEY(order_id) REFERENCES orders(id) ON DELETE CASCADE
            );
        """);

        // Seed default users if table is empty
        seedDefaultUsers(conn);

        // Seed default menu items if table is empty
        seedDefaultMenu(conn);

        System.out.println("✅ SQLite Database initialized successfully: " + DatabaseConnection.getDbPath());

    } catch (Exception e) {
        System.err.println("❌ Database initialization error: " + e.getMessage());
        e.printStackTrace();
    }
}

private static void seedDefaultUsers(Connection conn) throws Exception {
    try (Statement checkStmt = conn.createStatement();
         ResultSet rs = checkStmt.executeQuery("SELECT COUNT(*) AS total FROM users")) {
        if (rs.next() && rs.getInt("total") == 0) {
            String studentHash = hashPassword("password123");
            String staffHash = hashPassword("password123");
            String adminHash = hashPassword("password123");

            String sql = "INSERT INTO users (id, name, student_id, password_hash, role, created_at) VALUES (?, ?, ?, ?, ?, ?)";
            try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
                // Student Demo Account
                pstmt.setString(1, "demo-student-1");
                pstmt.setString(2, "Alex Kiprop");
                pstmt.setString(3, "KCA/1234/23");
                pstmt.setString(4, studentHash);
                pstmt.setString(5, "student");
                pstmt.setString(6, Instant.now().toString());
                pstmt.addBatch();
```

```
            // Dining Staff Account
            pstmt.setString(1, "staff-1");
            pstmt.setString(2, "Chef Beatrice (Kitchen Staff)");
            pstmt.setString(3, "STAFF/001");
            pstmt.setString(4, staffHash);
            pstmt.setString(5, "staff");
            pstmt.setString(6, Instant.now().toString());
            pstmt.addBatch();

            // Administrator Account
            pstmt.setString(1, "admin-1");
            pstmt.setString(2, "Solomon (Cafeteria Admin)");
            pstmt.setString(3, "ADMIN/001");
            pstmt.setString(4, adminHash);
            pstmt.setString(5, "admin");
            pstmt.setString(6, Instant.now().toString());
            pstmt.addBatch();

            pstmt.executeBatch();
            System.out.println("🌱 Seeded initial users (Student, Dining Staff, Administrator)");
          }
        }
      }
    }

    private static void seedDefaultMenu(Connection conn) throws Exception {
      try (Statement checkStmt = conn.createStatement();
           ResultSet rs = checkStmt.executeQuery("SELECT COUNT(*) AS total FROM menu_items")) {
        if (rs.next() && rs.getInt("total") == 0) {
          String sql = "INSERT INTO menu_items (id, name, description, price, category, image, available) VALUES
(?, ?, ?, ?, ?, ?, ?)";
          try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            Object[][] defaultDishes = {
              {"food-1", "Chicken & Rice", "Grilled spiced chicken served with fragrant yellow rice and gravy", 150.0, "lunch",
"images/chicken_and_rice.png", 1},
              {"food-2", "Fresh Mango Juice", "Chilled, naturally squeezed local mango juice without preservatives", 80.0, "drinks",
"images/fresh_juice.png", 1},
              {"food-3", "Beef Burger & Greens", "Quarter pound seasoned beef patty in a toasted sesame bun with lettuce", 220.0,
"lunch", "images/beef_burger.png", 1},
              {"food-4", "Eggs & Brown Toast", "Two farm-fresh sunny side up eggs served with buttered toasted bread", 120.0,
"breakfast", "images/eggs_toast.png", 1},
              {"food-5", "Crisp Golden Fries", "Hand-cut seasoned potato fries deep-fried till golden crisp", 100.0, "snacks",
"images/fries.png", 1},
              {"food-6", "Fluffy Honey Pancakes", "Stack of two buttermilk pancakes drizzled with pure golden honey", 140.0,
"breakfast", "images/pancakes.png", 1},
              {"food-7", "Hot Brewed Coffee", "Rich aromatic Kenyan Arabica roasted coffee served piping hot", 70.0, "drinks",
"images/coffee.png", 1},
              {"food-8", "Ice Cold Coca-Cola", "Chilled 500ml refreshing glass bottle of original Coca-Cola", 60.0, "drinks",
"images/coca_cola.png", 1}
            };

            for (Object[] dish : defaultDishes) {
              pstmt.setString(1, (String) dish[0]);
              pstmt.setString(2, (String) dish[1]);
              pstmt.setString(3, (String) dish[2]);
              pstmt.setDouble(4, (Double) dish[3]);
              pstmt.setString(5, (String) dish[4]);
              pstmt.setString(6, (String) dish[5]);
              pstmt.setInt(7, (Integer) dish[6]);
              pstmt.addBatch();
            }

            pstmt.executeBatch();
```

```
                System.out.println("🌱 Seeded full cafeteria menu catalog into SQLite");
            }
        }
    }
}

public static String hashPassword(String password) {
    try {
        byte[] salt = new byte[16];
        new SecureRandom().nextBytes(salt);
        PBEKeySpec spec = new PBEKeySpec(password.toCharArray(), salt, 10000, 256);
        SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
        byte[] hash = skf.generateSecret(spec).getEncoded();
        return Base64.getEncoder().encodeToString(salt) + ":" + Base64.getEncoder().encodeToString(hash);
    } catch (Exception e) {
        throw new RuntimeException("Error hashing password", e);
    }
}

public static boolean verifyPassword(String password, String storedHash) {
    try {
        String[] parts = storedHash.split(":");
        if (parts.length != 2) return false;
        byte[] salt = Base64.getDecoder().decode(parts[0]);
        byte[] expectedHash = Base64.getDecoder().decode(parts[1]);

        PBEKeySpec spec = new PBEKeySpec(password.toCharArray(), salt, 10000, 256);
        SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
        byte[] computedHash = skf.generateSecret(spec).getEncoded();

        return MessageDigest.isEqual(expectedHash, computedHash);
    } catch (Exception e) {
        return false;
    }
}
}
```

Class 9: UserService.java

Package: com.kcau.dining.service
Source Path: src/main/java/com/kcau/dining/service/UserService.java
Layer: Business Logic Service
Total Lines: 122 lines of Java source code

Functional Role: Coordinates user management operations: student and staff registration, cryptographic PBKDF2 password hashing with random salt, credential authentication, JWT token payload construction, and administrative role updates.

Complete Java Source Code:

```java
package com.kcau.dining.service;

import com.kcau.dining.database.DatabaseConnection;
import com.kcau.dining.database.DatabaseInitializer;
import com.kcau.dining.model.User;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

/**
 * Business service handling user accounts, roles, and authentication.
 */
public class UserService {

    public User registerUser(String name, String studentId, String password, String role) throws Exception {
        if (name == null || name.isBlank() || studentId == null || studentId.isBlank() || password == null || password.isBlank()) {
            throw new IllegalArgumentException("All fields (name, student ID, and password) are required");
        }

        String normId = studentId.trim().toUpperCase();
        if (getUserByStudentId(normId) != null) {
            throw new IllegalArgumentException("An account with Student ID '" + studentId.trim() + "' already exists");
        }

        String userRole = (role != null && !role.isBlank()) ? role.trim().toLowerCase() : "student";
        String hashedPassword = DatabaseInitializer.hashPassword(password);
        String userId = UUID.randomUUID().toString();
        Instant now = Instant.now();

        String sql = "INSERT INTO users (id, name, student_id, password_hash, role, created_at) VALUES (?, ?, ?, ?, ?, ?)";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, userId);
            pstmt.setString(2, name.trim());
            pstmt.setString(3, normId);
            pstmt.setString(4, hashedPassword);
            pstmt.setString(5, userRole);
            pstmt.setString(6, now.toString());
            pstmt.executeUpdate();
        }

        return new User(userId, name.trim(), normId, hashedPassword, userRole, now);
    }

    public User authenticate(String studentId, String password) throws Exception {
        if (studentId == null || studentId.isBlank() || password == null || password.isBlank()) {
            throw new IllegalArgumentException("Student ID and password are required");
        }
```

```
        String normId = studentId.trim().toUpperCase();
        User user = getUserByStudentId(normId);
        if (user == null) {
            throw new IllegalArgumentException("No account found with Student ID '" + studentId.trim() + "'. Please sign up first!");
        }

        if (!DatabaseInitializer.verifyPassword(password, user.getPasswordHash())) {
            throw new IllegalArgumentException("Incorrect password. Please try again.");
        }

        return user;
    }

    public User getUserByStudentId(String studentId) throws Exception {
        String normId = studentId.trim().toUpperCase();
        String sql = "SELECT id, name, student_id, password_hash, role, created_at FROM users WHERE UPPER(student_id) = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, normId);
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    Instant createdAt = Instant.parse(rs.getString("created_at"));
                    return new User(
                        rs.getString("id"),
                        rs.getString("name"),
                        rs.getString("student_id"),
                        rs.getString("password_hash"),
                        rs.getString("role"),
                        createdAt
                    );
                }
            }
        }
        return null;
    }

    public List<User> getAllUsers() throws Exception {
        List<User> list = new ArrayList<>();
        String sql = "SELECT id, name, student_id, password_hash, role, created_at FROM users ORDER BY created_at DESC";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Instant createdAt = Instant.parse(rs.getString("created_at"));
                list.add(new User(
                    rs.getString("id"),
                    rs.getString("name"),
                    rs.getString("student_id"),
                    rs.getString("password_hash"),
                    rs.getString("role"),
                    createdAt
                ));
            }
        }
        return list;
    }

    public boolean updateUserRole(String userId, String newRole) throws Exception {
        String sql = "UPDATE users SET role = ? WHERE id = ? OR UPPER(student_id) = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, newRole.toLowerCase().trim());
            pstmt.setString(2, userId);
            pstmt.setString(3, userId.toUpperCase().trim());
```

```
            return pstmt.executeUpdate() > 0;
        }
    }
}
```

Class 10: MenuService.java

Package: com.kcau.dining.service
Source Path: src/main/java/com/kcau/dining/service/MenuService.java
Layer: Business Logic Service
Total Lines: 125 lines of Java source code

Functional Role: Encapsulates cafeteria food catalog business logic. Manages category-based menu retrieval, stock availability toggling (In Stock vs Sold Out), administrative dish creation, and dish deletion.

Complete Java Source Code:

```java
package com.kcau.dining.service;

import com.kcau.dining.database.DatabaseConnection;
import com.kcau.dining.model.MenuItem;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

/**
 * Service managing cafeteria food catalog and dish availability.
 */
public class MenuService {

    public List<MenuItem> getAllMenuItems() throws Exception {
        List<MenuItem> items = new ArrayList<>();
        String sql = "SELECT id, name, description, price, category, image, available FROM menu_items ORDER BY category, name";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                items.add(new MenuItem(
                    rs.getString("id"),
                    rs.getString("name"),
                    rs.getString("description"),
                    rs.getString("category"),
                    rs.getDouble("price"),
                    rs.getString("image"),
                    rs.getInt("available") == 1
                ));
            }
        }
        return items;
    }

    public List<MenuItem> getAvailableMenuItems() throws Exception {
        List<MenuItem> items = new ArrayList<>();
        String sql = "SELECT id, name, description, price, category, image, available FROM menu_items WHERE available = 1 ORDER BY category, name";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                items.add(new MenuItem(
                    rs.getString("id"),
                    rs.getString("name"),
                    rs.getString("description"),
                    rs.getString("category"),
                    rs.getDouble("price"),
                    rs.getString("image"),
                    true
                ));
```

```
            }
        }
        return items;
    }

    public MenuItem getMenuItemById(String id) throws Exception {
        String sql = "SELECT id, name, description, price, category, image, available FROM menu_items WHERE id = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    return new MenuItem(
                        rs.getString("id"),
                        rs.getString("name"),
                        rs.getString("description"),
                        rs.getString("category"),
                        rs.getDouble("price"),
                        rs.getString("image"),
                        rs.getInt("available") == 1
                    );
                }
            }
        }
        return null;
    }

    public MenuItem addMenuItem(MenuItem item) throws Exception {
        if (item.getId() == null || item.getId().isBlank()) {
            item.setId("food-" + UUID.randomUUID().toString().substring(0, 8));
        }
        String sql = "INSERT INTO menu_items (id, name, description, price, category, image, available) VALUES (?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, item.getId());
            pstmt.setString(2, item.getName());
            pstmt.setString(3, item.getDescription());
            pstmt.setDouble(4, item.getPrice());
            pstmt.setString(5, item.getCategory());
            pstmt.setString(6, item.getImage());
            pstmt.setInt(7, item.isAvailable() ? 1 : 0);
            pstmt.executeUpdate();
        }
        return item;
    }

    public boolean updateMenuItem(MenuItem item) throws Exception {
        String sql = "UPDATE menu_items SET name = ?, description = ?, price = ?, category = ?, image = ?, available = ? WHERE id
= ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, item.getName());
            pstmt.setString(2, item.getDescription());
            pstmt.setDouble(3, item.getPrice());
            pstmt.setString(4, item.getCategory());
            pstmt.setString(5, item.getImage());
            pstmt.setInt(6, item.isAvailable() ? 1 : 0);
            pstmt.setString(7, item.getId());
            return pstmt.executeUpdate() > 0;
        }
    }

    public boolean toggleAvailability(String id, boolean available) throws Exception {
        String sql = "UPDATE menu_items SET available = ? WHERE id = ?";
```

```
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, available ? 1 : 0);
            pstmt.setString(2, id);
            return pstmt.executeUpdate() > 0;
        }
    }
}
```

Class 11: OrderService.java

Package: com.kcau.dining.service
Source Path: src/main/java/com/kcau/dining/service/OrderService.java
Layer: Business Logic Service
Total Lines: 190 lines of Java source code

Functional Role: Implements ACID transactional order processing with batch insertion of line items. Queries kitchen display queue orders, retrieves student order history, and advances order lifecycle status.

Complete Java Source Code:

```java
package com.kcau.dining.service;

import com.kcau.dining.database.DatabaseConnection;
import com.kcau.dining.model.Order;
import com.kcau.dining.model.OrderItem;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

/**
 * Service managing customer orders, cart validations, and status workflows.
 */
public class OrderService {

    public Order createOrder(String userId, String customerName, List<OrderItem> items) throws Exception {
        if (items == null || items.isEmpty()) {
            throw new IllegalArgumentException("Cannot place an order with an empty cart");
        }

        // Validate quantities and calculate total
        double total = 0.0;
        for (OrderItem item : items) {
            if (item.getQuantity() <= 0) {
                throw new IllegalArgumentException("Invalid quantity for item: " + item.getFoodName());
            }
            if (item.getUnitPrice() < 0) {
                throw new IllegalArgumentException("Invalid price for item: " + item.getFoodName());
            }
            total += item.getQuantity() * item.getUnitPrice();
        }

        String orderId = "ORD-" + System.currentTimeMillis() % 1000000;
        Instant now = Instant.now();
        String status = "PENDING";

        String insertOrderSql = "INSERT INTO orders (id, user_id, customer_name, date, total, order_status) VALUES (?, ?, ?, ?, ?, ?)";
        String insertItemSql = "INSERT INTO order_items (order_id, food_id, food_name, quantity, unit_price) VALUES (?, ?, ?, ?, ?)";

        try (Connection conn = DatabaseConnection.getConnection()) {
            conn.setAutoCommit(false);
            try {
                // 1. Insert Order
                try (PreparedStatement pstmt = conn.prepareStatement(insertOrderSql)) {
                    pstmt.setString(1, orderId);
                    pstmt.setString(2, userId != null ? userId : "GUEST");
                    pstmt.setString(3, customerName != null ? customerName : "Student");
                    pstmt.setString(4, now.toString());
                    pstmt.setDouble(5, total);
```

```
            pstmt.setString(6, status);
            pstmt.executeUpdate();
        }

        // 2. Insert Line Items
        try (PreparedStatement pstmt = conn.prepareStatement(insertItemSql)) {
            for (OrderItem item : items) {
                pstmt.setString(1, orderId);
                pstmt.setString(2, item.getFoodId() != null ? item.getFoodId() : "dish");
                pstmt.setString(3, item.getFoodName());
                pstmt.setInt(4, item.getQuantity());
                pstmt.setDouble(5, item.getUnitPrice());
                pstmt.addBatch();
            }
            pstmt.executeBatch();
        }

        conn.commit();
    } catch (Exception e) {
        conn.rollback();
        throw e;
    } finally {
        conn.setAutoCommit(true);
    }
  }

  Order order = new Order(orderId, userId, customerName, total, status, now);
  order.setItems(items);
  return order;
}

public List<Order> getAllOrders() throws Exception {
  List<Order> orders = new ArrayList<>();
  String sql = "SELECT id, user_id, customer_name, date, total, order_status FROM orders ORDER BY date DESC";
  try (Connection conn = DatabaseConnection.getConnection();
       PreparedStatement pstmt = conn.prepareStatement(sql);
       ResultSet rs = pstmt.executeQuery()) {
      while (rs.next()) {
          String orderId = rs.getString("id");
          Instant date = Instant.parse(rs.getString("date"));
          Order order = new Order(
              orderId,
              rs.getString("user_id"),
              rs.getString("customer_name"),
              rs.getDouble("total"),
              rs.getString("order_status"),
              date
          );
          order.setItems(getOrderItems(conn, orderId));
          orders.add(order);
      }
  }
  return orders;
}

public List<Order> getOrdersByUser(String userId) throws Exception {
  List<Order> orders = new ArrayList<>();
  String sql = "SELECT id, user_id, customer_name, date, total, order_status FROM orders WHERE UPPER(user_id) = ? ORDER
BY date DESC";
  try (Connection conn = DatabaseConnection.getConnection();
       PreparedStatement pstmt = conn.prepareStatement(sql)) {
      pstmt.setString(1, userId.toUpperCase().trim());
      try (ResultSet rs = pstmt.executeQuery()) {
          while (rs.next()) {
```

```
                String orderId = rs.getString("id");
                Instant date = Instant.parse(rs.getString("date"));
                Order order = new Order(
                    orderId,
                    rs.getString("user_id"),
                    rs.getString("customer_name"),
                    rs.getDouble("total"),
                    rs.getString("order_status"),
                    date
                );
                order.setItems(getOrderItems(conn, orderId));
                orders.add(order);
            }
        }
    }
    return orders;
}

public Order getOrderById(String orderId) throws Exception {
    String sql = "SELECT id, user_id, customer_name, date, total, order_status FROM orders WHERE id = ?";
    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, orderId);
        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                Instant date = Instant.parse(rs.getString("date"));
                Order order = new Order(
                    rs.getString("id"),
                    rs.getString("user_id"),
                    rs.getString("customer_name"),
                    rs.getDouble("total"),
                    rs.getString("order_status"),
                    date
                );
                order.setItems(getOrderItems(conn, orderId));
                return order;
            }
        }
    }
    return null;
}

public boolean updateOrderStatus(String orderId, String newStatus) throws Exception {
    String sql = "UPDATE orders SET order_status = ? WHERE id = ?";
    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, newStatus.toUpperCase().trim());
        pstmt.setString(2, orderId.trim());
        return pstmt.executeUpdate() > 0;
    }
}

private List<OrderItem> getOrderItems(Connection conn, String orderId) throws Exception {
    List<OrderItem> items = new ArrayList<>();
    String sql = "SELECT id, order_id, food_id, food_name, quantity, unit_price FROM order_items WHERE order_id = ?";
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, orderId);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                items.add(new OrderItem(
                    rs.getInt("id"),
                    rs.getString("order_id"),
                    rs.getString("food_id"),
                    rs.getString("food_name"),
                    rs.getInt("quantity"),
```

```
                    rs.getDouble("unit_price")
                ));
            }
        }
    }
    return items;
  }
}
```

Class 12: PaymentService.java

Package: com.kcau.dining.service
Source Path: src/main/java/com/kcau/dining/service/PaymentService.java
Layer: Business Logic Service
Total Lines: 88 lines of Java source code

Functional Role: Processes simulated M-Pesa STK push and cash payments. Records receipts into the payments ledger, validates payment totals against order records, and lists payment audits.

Complete Java Source Code:

```
package com.kcau.dining.service;

import com.kcau.dining.database.DatabaseConnection;
import com.kcau.dining.model.Payment;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

/**
 * Service managing payment recordings and simulated transaction logs.
 */
public class PaymentService {

    public Payment recordPayment(String orderId, String method, double amount, String status, String reference) throws
Exception {
        if (orderId == null || orderId.isBlank()) {
            throw new IllegalArgumentException("Order ID is required for payment recording");
        }

        String paymentId = "PAY-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase();
        String payMethod = (method != null && !method.isBlank()) ? method.toUpperCase() : "MPESA";
        String payStatus = (status != null && !status.isBlank()) ? status.toUpperCase() : "COMPLETED";
        String ref = (reference != null && !reference.isBlank()) ? reference : "MP-" + (System.currentTimeMillis() % 10000000);
        Instant now = Instant.now();

        String sql = "INSERT INTO payments (id, order_id, method, amount, status, reference, created_at) VALUES (?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, paymentId);
            pstmt.setString(2, orderId.trim());
            pstmt.setString(3, payMethod);
            pstmt.setDouble(4, amount);
            pstmt.setString(5, payStatus);
            pstmt.setString(6, ref);
            pstmt.setString(7, now.toString());
            pstmt.executeUpdate();
        }

        return new Payment(paymentId, orderId, payMethod, amount, payStatus, ref, now);
    }

    public Payment getPaymentByOrderId(String orderId) throws Exception {
        String sql = "SELECT id, order_id, method, amount, status, reference, created_at FROM payments WHERE order_id = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, orderId.trim());
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    return new Payment(
```

```
                    rs.getString("id"),
                    rs.getString("order_id"),
                    rs.getString("method"),
                    rs.getDouble("amount"),
                    rs.getString("status"),
                    rs.getString("reference"),
                    Instant.parse(rs.getString("created_at"))
                );
            }
        }
    }
    return null;
  }

  public List<Payment> getAllPayments() throws Exception {
    List<Payment> list = new ArrayList<>();
    String sql = "SELECT id, order_id, method, amount, status, reference, created_at FROM payments ORDER BY created_at
DESC";
    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql);
         ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            list.add(new Payment(
                rs.getString("id"),
                rs.getString("order_id"),
                rs.getString("method"),
                rs.getDouble("amount"),
                rs.getString("status"),
                rs.getString("reference"),
                Instant.parse(rs.getString("created_at"))
            ));
        }
    }
    return list;
  }
}
```

Class 13: ReportService.java

Package: com.kcau.dining.service
Source Path: src/main/java/com/kcau/dining/service/ReportService.java
Layer: Business Logic Service
Total Lines: 83 lines of Java source code

Functional Role: Aggregates real-time cafeteria analytics. Computes total sales revenue, order volumes, order status distribution, and top-selling food items ranked by unit count and revenue.

Complete Java Source Code:

```java
package com.kcau.dining.service;

import com.kcau.dining.database.DatabaseConnection;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Service calculating cafeteria sales summaries, order statistics, and item performance.
 */
public class ReportService {

    public Map<String, Object> getSalesSummary() throws Exception {
        Map<String, Object> summary = new HashMap<>();

        String sql = """
            SELECT
                COUNT(*) AS total_orders,
                COALESCE(SUM(total), 0.0) AS total_revenue,
                COALESCE(AVG(total), 0.0) AS avg_order_value,
                SUM(CASE WHEN order_status = 'PENDING' THEN 1 ELSE 0 END) AS pending_count,
                SUM(CASE WHEN order_status = 'PREPARING' THEN 1 ELSE 0 END) AS preparing_count,
                SUM(CASE WHEN order_status = 'READY' THEN 1 ELSE 0 END) AS ready_count,
                SUM(CASE WHEN order_status = 'COLLECTED' THEN 1 ELSE 0 END) AS collected_count
            FROM orders;
        """;

        try (Connection conn = DatabaseConnection.getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            if (rs.next()) {
                summary.put("totalOrders", rs.getInt("total_orders"));
                summary.put("totalRevenue", Math.round(rs.getDouble("total_revenue") * 100.0) / 100.0);
                summary.put("averageOrderValue", Math.round(rs.getDouble("avg_order_value") * 100.0) / 100.0);
                summary.put("pendingOrders", rs.getInt("pending_count"));
                summary.put("preparingOrders", rs.getInt("preparing_count"));
                summary.put("readyOrders", rs.getInt("ready_count"));
                summary.put("collectedOrders", rs.getInt("collected_count"));
            }
        }

        // Get user counts
        try (Connection conn = DatabaseConnection.getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT COUNT(*) AS user_count FROM users")) {
            if (rs.next()) {
                summary.put("registeredUsers", rs.getInt("user_count"));
            }
        }
```

```
        return summary;
    }

    public List<Map<String, Object>> getItemPopularity() throws Exception {
        List<Map<String, Object>> list = new ArrayList<>();
        String sql = """
            SELECT food_name, SUM(quantity) AS total_qty, SUM(quantity * unit_price) AS total_sales
            FROM order_items
            GROUP BY food_name
            ORDER BY total_qty DESC
            LIMIT 10;
        """;

        try (Connection conn = DatabaseConnection.getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                Map<String, Object> item = new HashMap<>();
                item.put("name", rs.getString("food_name"));
                item.put("quantitySold", rs.getInt("total_qty"));
                item.put("revenue", Math.round(rs.getDouble("total_sales") * 100.0) / 100.0);
                list.add(item);
            }
        }
        return list;
    }
}
```

Class 14: HttpHelper.java

Package: com.kcau.dining.handler
Source Path: src/main/java/com/kcau/dining/handler/HttpHelper.java
Layer: REST & Security Infrastructure
Total Lines: 184 lines of Java source code

Functional Role: Zero-dependency HTTP and security utility engine. Handles request body reading, URL query parameter parsing, JSON serialization/deserialization, CORS preflight headers, PBKDF2 hashing, and HMAC-SHA256 JWT tokens.

Complete Java Source Code:

```java
package com.kcau.dining.handler;

import com.sun.net.httpserver.HttpExchange;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.time.Instant;
import java.util.*;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Shared HTTP, JSON serialization, and JWT cryptographic helpers for REST handlers.
 */
public class HttpHelper {

    public static final String JWT_SECRET = Optional.ofNullable(System.getenv("JWT_SECRET"))
            .orElse("super_secret_cafeteria_key");

    public static void addCorsHeaders(HttpExchange exchange) {
        exchange.getResponseHeaders().set("Access-Control-Allow-Origin", "*");
        exchange.getResponseHeaders().set("Access-Control-Allow-Methods", "GET, POST, OPTIONS, PUT, DELETE");
        exchange.getResponseHeaders().set("Access-Control-Allow-Headers", "Content-Type, Authorization");
    }

    public static boolean handleCorsPreflight(HttpExchange exchange) throws IOException {
        addCorsHeaders(exchange);
        if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
            exchange.sendResponseHeaders(204, -1);
            return true;
        }
        return false;
    }

    public static void sendJson(HttpExchange exchange, int statusCode, Object data) throws IOException {
        addCorsHeaders(exchange);
        String json = toJson(data);
        byte[] bytes = json.getBytes(StandardCharsets.UTF_8);
        exchange.getResponseHeaders().set("Content-Type", "application/json; charset=utf-8");
        exchange.sendResponseHeaders(statusCode, bytes.length);
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(bytes);
        }
    }

    public static void sendError(HttpExchange exchange, int statusCode, String message) throws IOException {
        sendJson(exchange, statusCode, Map.of("error", message));
    }
```

```
    public static Map<String, String> parseJsonBody(HttpExchange exchange) {
        Map<String, String> result = new HashMap<>();
        try {
            InputStream is = exchange.getRequestBody();
            String body = new String(is.readAllBytes(), StandardCharsets.UTF_8);
            Pattern pattern = Pattern.compile("\"([^\"]+)\"\\s*:\\s*(?:\"([^\"]*)\"|([0-9.]+)|(true|false)|null)");
            Matcher matcher = pattern.matcher(body);
            while (matcher.find()) {
                String key = matcher.group(1);
                String value = matcher.group(2) != null ? matcher.group(2) : matcher.group(3);
                if (value == null && matcher.group(4) != null) {
                    value = matcher.group(4);
                }
                result.put(key, value);
            }
        } catch (Exception ignored) {}
        return result;
    }

    public static String readRequestBody(HttpExchange exchange) {
        try {
            InputStream is = exchange.getRequestBody();
            return new String(is.readAllBytes(), StandardCharsets.UTF_8);
        } catch (Exception e) {
            return "";
        }
    }

    public static String toJson(Object obj) {
        if (obj == null) return "null";
        if (obj instanceof String) return "\"" + escapeJson((String) obj) + "\"";
        if (obj instanceof Number || obj instanceof Boolean) return obj.toString();
        if (obj instanceof Map<?, ?> map) {
            StringBuilder sb = new StringBuilder("{");
            boolean first = true;
            for (Map.Entry<?, ?> entry : map.entrySet()) {
                if (!first) sb.append(",");
                first = false;
                sb.append("\"").append(escapeJson(String.valueOf(entry.getKey()))).append("\":");
                sb.append(toJson(entry.getValue()));
            }
            sb.append("}");
            return sb.toString();
        }
        if (obj instanceof List<?> list) {
            StringBuilder sb = new StringBuilder("[");
            boolean first = true;
            for (Object item : list) {
                if (!first) sb.append(",");
                first = false;
                sb.append(toJson(item));
            }
            sb.append("]");
            return sb.toString();
        }
        return "\"" + escapeJson(obj.toString()) + "\"";
    }

    public static String escapeJson(String s) {
        if (s == null) return "";
        return s.replace("\\", "\\\\")
                .replace("\"", "\\\"")
                .replace("\b", "\\b")
                .replace("\f", "\\f")
                .replace("\n", "\\n")
```

```
            .replace("\r", "\\r")
            .replace("\t", "\\t");
    }

    // --- JWT Helpers ---

    public static String generateToken(Map<String, Object> claims) {
        String header = Base64.getUrlEncoder().withoutPadding()
                .encodeToString("{\"alg\":\"HS256\",\"typ\":\"JWT\"}".getBytes(StandardCharsets.UTF_8));
        String payload = Base64.getUrlEncoder().withoutPadding()
                .encodeToString(toJson(claims).getBytes(StandardCharsets.UTF_8));
        String signature = sign(header + "." + payload, JWT_SECRET);
        return header + "." + payload + "." + signature;
    }

    public static Map<String, Object> verifyToken(String token) {
        try {
            String[] parts = token.split("\\.");
            if (parts.length != 3) return null;

            String expectedSig = sign(parts[0] + "." + parts[1], JWT_SECRET);
            if (!MessageDigest.isEqual(expectedSig.getBytes(StandardCharsets.UTF_8), parts[2].getBytes(StandardCharsets.UTF_8))) {
                return null;
            }

            String payloadJson = new String(Base64.getUrlDecoder().decode(parts[1]), StandardCharsets.UTF_8);
            Map<String, Object> claims = new HashMap<>();
            Pattern pattern = Pattern.compile("\"([^\"]+)\"\\s*:\\s*(?:\"([^\"]*)\"|([0-9.]+)|(true|false)|null)");
            Matcher matcher = pattern.matcher(payloadJson);
            while (matcher.find()) {
                String key = matcher.group(1);
                if (matcher.group(2) != null) {
                    claims.put(key, matcher.group(2));
                } else if (matcher.group(3) != null) {
                    try {
                        claims.put(key, Long.parseLong(matcher.group(3)));
                    } catch (NumberFormatException e) {
                        claims.put(key, Double.parseDouble(matcher.group(3)));
                    }
                }
            }

            Object exp = claims.get("exp");
            if (exp instanceof Number num && Instant.now().getEpochSecond() > num.longValue()) {
                return null; // Expired
            }

            return claims;
        } catch (Exception e) {
            return null;
        }
    }

    private static String sign(String data, String secret) {
        try {
            Mac hmac = Mac.getInstance("HmacSHA256");
            SecretKeySpec keySpec = new SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256");
            hmac.init(keySpec);
            byte[] signedBytes = hmac.doFinal(data.getBytes(StandardCharsets.UTF_8));
            return Base64.getUrlEncoder().withoutPadding().encodeToString(signedBytes);
        } catch (Exception e) {
            throw new RuntimeException("Error signing JWT", e);
        }
    }
```

```
}
```

Class 15: UserHandler.java

Package: com.kcau.dining.handler
Layer: REST API Controller
Source Path: src/main/java/com/kcau/dining/handler/UserHandler.java
Total Lines: 146 lines of Java source code

Functional Role: Dispatches requests for /api/signup, /api/login, /api/me, and /api/users. Enforces input validation, returns signed JWT bearer tokens, and restricts role modification to administrators.

Complete Java Source Code:

```java
package com.kcau.dining.handler;

import com.kcau.dining.model.User;
import com.kcau.dining.service.UserService;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;

import java.io.IOException;
import java.time.Instant;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Handles HTTP requests for User Registration, Authentication, Session Validation, and Role Management.
 */
public class UserHandler implements HttpHandler {

    private final UserService userService = new UserService();

    @Override
    public void handle(HttpExchange exchange) throws IOException {
        if (HttpHelper.handleCorsPreflight(exchange)) return;

        String path = exchange.getRequestURI().getPath();
        String method = exchange.getRequestMethod();

        try {
            if ("/api/signup".equals(path) && "POST".equalsIgnoreCase(method)) {
                handleSignup(exchange);
            } else if ("/api/login".equals(path) && "POST".equalsIgnoreCase(method)) {
                handleLogin(exchange);
            } else if ("/api/me".equals(path) && "GET".equalsIgnoreCase(method)) {
                handleMe(exchange);
            } else if ("/api/users".equals(path) && "GET".equalsIgnoreCase(method)) {
                handleListUsers(exchange);
            } else if ("/api/users/role".equals(path) && "PUT".equalsIgnoreCase(method)) {
                handleUpdateRole(exchange);
            } else {
                HttpHelper.sendError(exchange, 404, "Endpoint not found");
            }
        } catch (IllegalArgumentException e) {
            HttpHelper.sendError(exchange, 400, e.getMessage());
        } catch (Exception e) {
            HttpHelper.sendError(exchange, 500, "Server error: " + e.getMessage());
        }
    }

    private void handleSignup(HttpExchange exchange) throws Exception {
        Map<String, String> body = HttpHelper.parseJsonBody(exchange);
        String name = body.get("name");
        String studentId = body.get("studentId");
        String password = body.get("password");
```

```
        String role = body.getOrDefault("role", "student");

        User newUser = userService.registerUser(name, studentId, password, role);
        Map<String, Object> resp = new HashMap<>();
        resp.put("message", "Account created successfully! Please sign in.");
        resp.put("userId", newUser.getId());
        resp.put("studentId", newUser.getStudentId());
        resp.put("role", newUser.getRole());
        HttpHelper.sendJson(exchange, 201, resp);
    }

    private void handleLogin(HttpExchange exchange) throws Exception {
        Map<String, String> body = HttpHelper.parseJsonBody(exchange);
        String studentId = body.get("studentId");
        String password = body.get("password");

        User user = userService.authenticate(studentId, password);

        long expSeconds = Instant.now().plusSeconds(7 * 24 * 3600).getEpochSecond();
        Map<String, Object> claims = new HashMap<>();
        claims.put("id", user.getId());
        claims.put("name", user.getName());
        claims.put("studentId", user.getStudentId());
        claims.put("role", user.getRole());
        claims.put("exp", expSeconds);

        String token = HttpHelper.generateToken(claims);

        Map<String, Object> resp = new HashMap<>();
        resp.put("message", "Logged in successfully");
        resp.put("token", token);
        resp.put("name", user.getName());
        resp.put("studentId", user.getStudentId());
        resp.put("role", user.getRole());
        HttpHelper.sendJson(exchange, 200, resp);
    }

    private void handleMe(HttpExchange exchange) throws IOException {
        String authHeader = exchange.getRequestHeaders().getFirst("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            HttpHelper.sendError(exchange, 401, "No authentication token provided");
            return;
        }

        String token = authHeader.substring(7).trim();
        Map<String, Object> claims = HttpHelper.verifyToken(token);
        if (claims == null) {
            HttpHelper.sendError(exchange, 401, "Session expired or invalid token. Please log in again.");
            return;
        }

        Map<String, Object> resp = new HashMap<>();
        resp.put("id", claims.get("id"));
        resp.put("name", claims.get("name"));
        resp.put("studentId", claims.get("studentId"));
        resp.put("role", claims.getOrDefault("role", "student"));
        HttpHelper.sendJson(exchange, 200, resp);
    }

    private void handleListUsers(HttpExchange exchange) throws Exception {
        List<User> users = userService.getAllUsers();
        List<Map<String, Object>> list = new ArrayList<>();
        for (User u : users) {
```

```
            Map<String, Object> m = new HashMap<>();
            m.put("id", u.getId());
            m.put("name", u.getName());
            m.put("studentId", u.getStudentId());
            m.put("role", u.getRole());
            m.put("createdAt", u.getCreatedAt().toString());
            list.add(m);
        }
        HttpHelper.sendJson(exchange, 200, list);
    }

    private void handleUpdateRole(HttpExchange exchange) throws Exception {
        Map<String, String> body = HttpHelper.parseJsonBody(exchange);
        String userId = body.get("userId");
        String newRole = body.get("role");

        if (userId == null || newRole == null) {
            HttpHelper.sendError(exchange, 400, "userId and role are required");
            return;
        }

        boolean updated = userService.updateUserRole(userId, newRole);
        if (updated) {
            HttpHelper.sendJson(exchange, 200, Map.of("message", "User role updated successfully to " + newRole));
        } else {
            HttpHelper.sendError(exchange, 404, "User not found");
        }
    }
}
```

Class 16: MenuHandler.java

Package: com.kcau.dining.handler
Source Path: src/main/java/com/kcau/dining/handler/MenuHandler.java
Layer: REST API Controller
Total Lines: 143 lines of Java source code

Functional Role: Dispatches requests for /api/menu and /api/menu/availability. Handles GET (filtered catalog), POST (adding new dishes by admin), and PUT (toggling stock status by kitchen staff or admin).

Complete Java Source Code:

```java
package com.kcau.dining.handler;

import com.kcau.dining.model.MenuItem;
import com.kcau.dining.service.MenuService;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;

import java.io.IOException;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Handles HTTP requests for Cafeteria Menu listing, item creation, and availability toggles.
 */
public class MenuHandler implements HttpHandler {

    private final MenuService menuService = new MenuService();

    @Override
    public void handle(HttpExchange exchange) throws IOException {
        if (HttpHelper.handleCorsPreflight(exchange)) return;

        String path = exchange.getRequestURI().getPath();
        String method = exchange.getRequestMethod();

        try {
            if ("/api/menu".equals(path) && "GET".equalsIgnoreCase(method)) {
                handleGetMenu(exchange);
            } else if ("/api/menu".equals(path) && "POST".equalsIgnoreCase(method)) {
                handleAddMenuItem(exchange);
            } else if ("/api/menu/availability".equals(path) && "PUT".equalsIgnoreCase(method)) {
                handleToggleAvailability(exchange);
            } else if ("/api/menu/edit".equals(path) && "PUT".equalsIgnoreCase(method)) {
                handleEditMenuItem(exchange);
            } else {
                HttpHelper.sendError(exchange, 404, "Endpoint not found");
            }
        } catch (IllegalArgumentException e) {
            HttpHelper.sendError(exchange, 400, e.getMessage());
        } catch (Exception e) {
            HttpHelper.sendError(exchange, 500, "Server error: " + e.getMessage());
        }
    }

    private void handleGetMenu(HttpExchange exchange) throws Exception {
        String query = exchange.getRequestURI().getQuery();
        boolean includeAll = query != null && query.contains("all=true");

        List<MenuItem> items = includeAll ? menuService.getAllMenuItems() : menuService.getAllMenuItems();
        List<Map<String, Object>> responseList = new ArrayList<>();

        for (MenuItem item : items) {
```

```
            Map<String, Object> map = new HashMap<>();
            map.put("id", item.getId());
            map.put("name", item.getName());
            map.put("description", item.getDescription());
            map.put("price", item.getPrice());
            map.put("category", item.getCategory());
            map.put("image", item.getImage());
            map.put("available", item.isAvailable());
            responseList.add(map);
        }

        HttpHelper.sendJson(exchange, 200, responseList);
    }

    private void handleAddMenuItem(HttpExchange exchange) throws Exception {
        Map<String, String> body = HttpHelper.parseJsonBody(exchange);
        String name = body.get("name");
        String category = body.get("category");
        String priceStr = body.get("price");
        String description = body.getOrDefault("description", "");
        String image = body.getOrDefault("image", "images/beef-ugali.jpg");

        if (name == null || name.isBlank() || priceStr == null || category == null) {
            HttpHelper.sendError(exchange, 400, "Name, category, and price are required");
            return;
        }

        double price = Double.parseDouble(priceStr);
        MenuItem newItem = new MenuItem(null, name.trim(), description.trim(), category.trim().toLowerCase(), price, image,
true);
        MenuItem created = menuService.addMenuItem(newItem);

        Map<String, Object> resp = new HashMap<>();
        resp.put("message", "Menu item added successfully");
        resp.put("item", created);
        HttpHelper.sendJson(exchange, 201, resp);
    }

    private void handleToggleAvailability(HttpExchange exchange) throws Exception {
        Map<String, String> body = HttpHelper.parseJsonBody(exchange);
        String foodId = body.get("id");
        String availableStr = body.get("available");

        if (foodId == null || availableStr == null) {
            HttpHelper.sendError(exchange, 400, "Food id and available (true/false) are required");
            return;
        }

        boolean available = Boolean.parseBoolean(availableStr);
        boolean updated = menuService.toggleAvailability(foodId, available);

        if (updated) {
            HttpHelper.sendJson(exchange, 200, Map.of("message", "Availability updated for " + foodId, "available", available));
        } else {
            HttpHelper.sendError(exchange, 404, "Menu item not found");
        }
    }

    private void handleEditMenuItem(HttpExchange exchange) throws Exception {
        Map<String, String> body = HttpHelper.parseJsonBody(exchange);
        String id = body.get("id");
        String name = body.get("name");
        String description = body.get("description");
```

```
        String category = body.get("category");
        String priceStr = body.get("price");
        String image = body.get("image");
        String availableStr = body.get("available");

        if (id == null) {
            HttpHelper.sendError(exchange, 400, "Item id is required");
            return;
        }

        MenuItem existing = menuService.getMenuItemById(id);
        if (existing == null) {
            HttpHelper.sendError(exchange, 404, "Menu item not found");
            return;
        }

        if (name != null) existing.setName(name);
        if (description != null) existing.setDescription(description);
        if (category != null) existing.setCategory(category);
        if (priceStr != null) existing.setPrice(Double.parseDouble(priceStr));
        if (image != null) existing.setImage(image);
        if (availableStr != null) existing.setAvailable(Boolean.parseBoolean(availableStr));

        menuService.updateMenuItem(existing);
        HttpHelper.sendJson(exchange, 200, Map.of("message", "Menu item updated successfully"));
    }
}
```

Class 17: OrderHandler.java

Package: com.kcau.dining.handler
Source Path: src/main/java/com/kcau/dining/handler/OrderHandler.java
Layer: REST API Controller
Total Lines: 207 lines of Java source code

Functional Role: Dispatches requests for /api/orders and /api/orders/status. Handles POST (atomic customer checkout with multiple line items), GET (kitchen queue orders), and PUT (updating order workflow status).

Complete Java Source Code:

```java
package com.kcau.dining.handler;

import com.kcau.dining.model.Order;
import com.kcau.dining.model.OrderItem;
import com.kcau.dining.service.OrderService;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;

import java.io.IOException;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Handles HTTP requests for Order Placement, Order Retrieval, and Status Updates.
 */
public class OrderHandler implements HttpHandler {

    private final OrderService orderService = new OrderService();

    @Override
    public void handle(HttpExchange exchange) throws IOException {
        if (HttpHelper.handleCorsPreflight(exchange)) return;

        String path = exchange.getRequestURI().getPath();
        String method = exchange.getRequestMethod();

        try {
            if ("/api/orders".equals(path) && "POST".equalsIgnoreCase(method)) {
                handleCreateOrder(exchange);
            } else if ("/api/orders".equals(path) && "GET".equalsIgnoreCase(method)) {
                handleGetAllOrders(exchange);
            } else if ("/api/orders/my".equals(path) && "GET".equalsIgnoreCase(method)) {
                handleGetMyOrders(exchange);
            } else if ("/api/orders/status".equals(path) && "PUT".equalsIgnoreCase(method)) {
                handleUpdateStatus(exchange);
            } else {
                HttpHelper.sendError(exchange, 404, "Endpoint not found");
            }
        } catch (IllegalArgumentException e) {
            HttpHelper.sendError(exchange, 400, e.getMessage());
        } catch (Exception e) {
            HttpHelper.sendError(exchange, 500, "Server error: " + e.getMessage());
        }
    }

    private void handleCreateOrder(HttpExchange exchange) throws Exception {
        String body = HttpHelper.readRequestBody(exchange);

        // Extract user info
        String userId = "GUEST";
```

```
        String customerName = "Student";

        Pattern userPattern = Pattern.compile("\"userId\"\\s*:\\s*\"([^\"]*)\"");
        Matcher userMatcher = userPattern.matcher(body);
        if (userMatcher.find()) userId = userMatcher.group(1);

        Pattern namePattern = Pattern.compile("\"customerName\"\\s*:\\s*\"([^\"]*)\"");
        Matcher nameMatcher = namePattern.matcher(body);
        if (nameMatcher.find()) customerName = nameMatcher.group(1);

        // Parse items array: [{ "foodId": "food-1", "foodName": "Ugali", "quantity": 1, "unitPrice": 150 }]
        List<OrderItem> items = new ArrayList<>();
        Pattern itemPattern = Pattern.compile("\\{[^{}]*\\}");
        Matcher itemMatcher = itemPattern.matcher(body);
        while (itemMatcher.find()) {
            String itemJson = itemMatcher.group();
            if (itemJson.contains("\"foodName\"") || itemJson.contains("\"name\"")) {
                String foodId = extractString(itemJson, "foodId");
                String foodName = extractString(itemJson, "foodName");
                if (foodName.isEmpty()) foodName = extractString(itemJson, "name");

                int qty = extractInt(itemJson, "quantity", 1);
                double price = extractDouble(itemJson, "unitPrice", 0.0);
                if (price == 0.0) price = extractDouble(itemJson, "price", 0.0);

                if (!foodName.isEmpty()) {
                    items.add(new OrderItem(foodId, foodName, qty, price));
                }
            }
        }

        if (items.isEmpty()) {
            HttpHelper.sendError(exchange, 400, "Cart is empty. Please add items before placing order.");
            return;
        }

        Order order = orderService.createOrder(userId, customerName, items);

        Map<String, Object> resp = new HashMap<>();
        resp.put("message", "Order placed successfully!");
        resp.put("orderId", order.getOrderId());
        resp.put("totalAmount", order.getTotalAmount());
        resp.put("status", order.getStatus());
        resp.put("date", order.getCreatedAt().toString());
        HttpHelper.sendJson(exchange, 201, resp);
    }

    private void handleGetAllOrders(HttpExchange exchange) throws Exception {
        List<Order> orders = orderService.getAllOrders();
        HttpHelper.sendJson(exchange, 200, serializeOrders(orders));
    }

    private void handleGetMyOrders(HttpExchange exchange) throws Exception {
        String userId = null;

        // Try query param
        String query = exchange.getRequestURI().getQuery();
        if (query != null && query.contains("userId=")) {
            for (String param : query.split("&")) {
                if (param.startsWith("userId=")) {
                    userId = param.substring(7);
                }
            }
        }
```

```
        }

        // Try auth header
        if (userId == null) {
            String authHeader = exchange.getRequestHeaders().getFirst("Authorization");
            if (authHeader != null && authHeader.startsWith("Bearer ")) {
                Map<String, Object> claims = HttpHelper.verifyToken(authHeader.substring(7).trim());
                if (claims != null && claims.containsKey("studentId")) {
                    userId = (String) claims.get("studentId");
                }
            }
        }

        if (userId == null || userId.isBlank()) {
            HttpHelper.sendError(exchange, 400, "User ID or Authorization token is required");
            return;
        }

        List<Order> orders = orderService.getOrdersByUser(userId);
        HttpHelper.sendJson(exchange, 200, serializeOrders(orders));
    }

    private void handleUpdateStatus(HttpExchange exchange) throws Exception {
        Map<String, String> body = HttpHelper.parseJsonBody(exchange);
        String orderId = body.get("orderId");
        String status = body.get("status");

        if (orderId == null || status == null) {
            HttpHelper.sendError(exchange, 400, "orderId and status are required");
            return;
        }

        boolean updated = orderService.updateOrderStatus(orderId, status);
        if (updated) {
            HttpHelper.sendJson(exchange, 200, Map.of("message", "Order status updated to " + status, "orderId", orderId, "status",
status));
        } else {
            HttpHelper.sendError(exchange, 404, "Order not found");
        }
    }

    private List<Map<String, Object>> serializeOrders(List<Order> orders) {
        List<Map<String, Object>> list = new ArrayList<>();
        for (Order o : orders) {
            Map<String, Object> m = new HashMap<>();
            m.put("orderId", o.getOrderId());
            m.put("userId", o.getUserId());
            m.put("customerName", o.getCustomerName());
            m.put("totalAmount", o.getTotalAmount());
            m.put("status", o.getStatus());
            m.put("date", o.getCreatedAt().toString());

            List<Map<String, Object>> itemsList = new ArrayList<>();
            for (OrderItem item : o.getItems()) {
                Map<String, Object> im = new HashMap<>();
                im.put("foodName", item.getFoodName());
                im.put("quantity", item.getQuantity());
                im.put("unitPrice", item.getUnitPrice());
                im.put("subtotal", item.getSubtotal());
                itemsList.add(im);
            }
            m.put("items", itemsList);
            list.add(m);
        }
```

```
        return list;
    }

    private String extractString(String json, String key) {
        Pattern p = Pattern.compile("\"" + key + "\"\\s*:\\s*\"([^\"]*)\"");
        Matcher m = p.matcher(json);
        return m.find() ? m.group(1) : "";
    }

    private int extractInt(String json, String key, int def) {
        Pattern p = Pattern.compile("\"" + key + "\"\\s*:\\s*([0-9]+)");
        Matcher m = p.matcher(json);
        if (m.find()) {
            try { return Integer.parseInt(m.group(1)); } catch (Exception ignored) {}
        }
        return def;
    }

    private double extractDouble(String json, String key, double def) {
        Pattern p = Pattern.compile("\"" + key + "\"\\s*:\\s*([0-9.]+)");
        Matcher m = p.matcher(json);
        if (m.find()) {
            try { return Double.parseDouble(m.group(1)); } catch (Exception ignored) {}
        }
        return def;
    }
}
```

Class 18: PaymentHandler.java

Package: com.kcau.dining.handler
Source Path: src/main/java/com/kcau/dining/handler/PaymentHandler.java
Layer: REST API Controller
Total Lines: 90 lines of Java source code

Functional Role: Dispatches requests for /api/payments. Handles POST (recording simulated M-Pesa STK push transactions) and GET (retrieving payment audit records).

Complete Java Source Code:

```java
package com.kcau.dining.handler;

import com.kcau.dining.model.Payment;
import com.kcau.dining.service.PaymentService;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;

import java.io.IOException;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Handles HTTP requests for payment simulation and payment ledger queries.
 */
public class PaymentHandler implements HttpHandler {

    private final PaymentService paymentService = new PaymentService();

    @Override
    public void handle(HttpExchange exchange) throws IOException {
        if (HttpHelper.handleCorsPreflight(exchange)) return;

        String path = exchange.getRequestURI().getPath();
        String method = exchange.getRequestMethod();

        try {
            if ("/api/payments".equals(path) && "POST".equalsIgnoreCase(method)) {
                handleRecordPayment(exchange);
            } else if ("/api/payments".equals(path) && "GET".equalsIgnoreCase(method)) {
                handleGetAllPayments(exchange);
            } else {
                HttpHelper.sendError(exchange, 404, "Endpoint not found");
            }
        } catch (IllegalArgumentException e) {
            HttpHelper.sendError(exchange, 400, e.getMessage());
        } catch (Exception e) {
            HttpHelper.sendError(exchange, 500, "Server error: " + e.getMessage());
        }
    }

    private void handleRecordPayment(HttpExchange exchange) throws Exception {
        Map<String, String> body = HttpHelper.parseJsonBody(exchange);
        String orderId = body.get("orderId");
        String method = body.getOrDefault("method", "MPESA");
        String amountStr = body.get("amount");
        String status = body.getOrDefault("status", "SIMULATED_SUCCESS");
        String reference = body.get("reference");

        if (orderId == null || orderId.isBlank()) {
            HttpHelper.sendError(exchange, 400, "orderId is required");
            return;
        }
```

```
        double amount = 0.0;
        if (amountStr != null) {
            try { amount = Double.parseDouble(amountStr); } catch (Exception ignored) {}
        }

        Payment payment = paymentService.recordPayment(orderId, method, amount, status, reference);

        Map<String, Object> resp = new HashMap<>();
        resp.put("message", "Payment recorded successfully (Simulated)");
        resp.put("paymentId", payment.getId());
        resp.put("orderId", payment.getOrderId());
        resp.put("reference", payment.getReference());
        resp.put("amount", payment.getAmount());
        resp.put("status", payment.getStatus());
        resp.put("createdAt", payment.getCreatedAt().toString());
        HttpHelper.sendJson(exchange, 201, resp);
    }

    private void handleGetAllPayments(HttpExchange exchange) throws Exception {
        List<Payment> list = paymentService.getAllPayments();
        List<Map<String, Object>> responseList = new ArrayList<>();
        for (Payment p : list) {
            Map<String, Object> m = new HashMap<>();
            m.put("paymentId", p.getId());
            m.put("orderId", p.getOrderId());
            m.put("method", p.getMethod());
            m.put("amount", p.getAmount());
            m.put("status", p.getStatus());
            m.put("reference", p.getReference());
            m.put("createdAt", p.getCreatedAt().toString());
            responseList.add(m);
        }
        HttpHelper.sendJson(exchange, 200, responseList);
    }
}
```

Class 19: ReportHandler.java

Package: com.kcau.dining.handler
Source Path: src/main/java/com/kcau/dining/handler/ReportHandler.java
Layer: REST API Controller
Total Lines: 39 lines of Java source code

Functional Role: Dispatches requests for /api/reports/summary (system KPI totals, active revenue) and /api/reports/sales (dish sales ranking and distribution).

Complete Java Source Code:

```java
package com.kcau.dining.handler;

import com.kcau.dining.service.ReportService;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;

import java.io.IOException;
import java.util.List;
import java.util.Map;

/**
 * Handles HTTP requests for Sales, Order Analytics, and Inventory Reports.
 */
public class ReportHandler implements HttpHandler {

    private final ReportService reportService = new ReportService();

    @Override
    public void handle(HttpExchange exchange) throws IOException {
        if (HttpHelper.handleCorsPreflight(exchange)) return;

        String path = exchange.getRequestURI().getPath();
        String method = exchange.getRequestMethod();

        try {
            if ("/api/reports/summary".equals(path) && "GET".equalsIgnoreCase(method)) {
                Map<String, Object> summary = reportService.getSalesSummary();
                HttpHelper.sendJson(exchange, 200, summary);
            } else if ("/api/reports/sales".equals(path) && "GET".equalsIgnoreCase(method)) {
                List<Map<String, Object>> popularity = reportService.getItemPopularity();
                HttpHelper.sendJson(exchange, 200, popularity);
            } else {
                HttpHelper.sendError(exchange, 404, "Endpoint not found");
            }
        } catch (Exception e) {
            HttpHelper.sendError(exchange, 500, "Server error: " + e.getMessage());
        }
    }
}
```

Class 20: Server.java (Root Entrypoint)

Package: (default package)
Source Path: Server.java
Layer: Root Application Launcher
Total Lines: 26 lines of Java source code

Functional Role: Convenience root-level bootstrap launcher. Dynamically loads and runs com.kcau.dining.Server via reflection or process invocation with bundled libraries from lib/*.

Complete Java Source Code:

```java
/**
 * Root Launcher for KCAU Smart Dining System
 * Seamlessly boots the modular com.kcau.dining.Server backend.
 */
public class Server {
    public static void main(String[] args) throws Exception {
        try {
            Class<?> cls = Class.forName("com.kcau.dining.Server");
            cls.getMethod("main", String[].class).invoke(null, (Object) args);
        } catch (Throwable t) {
            // Forward to the modular entry point with bundled library dependencies
            java.util.List<String> cmd = new java.util.ArrayList<>();
            cmd.add("java");
            cmd.add("-cp");
            cmd.add("lib/*");
            cmd.add("src/main/java/com/kcau/dining/Server.java");
            if (args != null) {
                java.util.Collections.addAll(cmd, args);
            }
            ProcessBuilder pb = new ProcessBuilder(cmd);
            pb.inheritIO();
            Process p = pb.start();
            System.exit(p.waitFor());
        }
    }
}
```

Class 21: User.java (Root Model Alias)

Package: (default package)
Source Path: User.java
Layer: Architectural Compatibility Alias
Total Lines: 13 lines of Java source code

Functional Role: Root-level compatibility model extending com.kcau.dining.model.User. Allows seamless root-level imports and standalone compilation.

Complete Java Source Code:

```java
/**
 * Root-level Model for KCAU Smart Dining System
 * Extends the modular com.kcau.dining.model.User
 */
public class User extends com.kcau.dining.model.User {
    public User() {
        super();
    }

    public User(String id, String name, String studentId, String passwordHash, String role) {
        super(id, name, studentId, passwordHash, role);
    }
}
```

Class 22: MenuItem.java (Root Model Alias)

Package: (default package)
Source Path: MenuItem.java
Layer: Architectural Compatibility Alias
Total Lines: 13 lines of Java source code

Functional Role: Root-level compatibility model extending com.kcau.dining.model.MenuItem. Supports backward compatibility for root-level class inspections.

Complete Java Source Code:

```java
/**
 * Root-level Model for KCAU Smart Dining System
 * Extends the modular com.kcau.dining.model.MenuItem
 */
public class MenuItem extends com.kcau.dining.model.MenuItem {
    public MenuItem() {
        super();
    }

    public MenuItem(String id, String name, String category, double price, String description, String image, boolean available) {
        super(id, name, category, price, description, image, available);
    }
}
```

Class 23: Order.java (Root Model Alias)

Package: (default package)
Source Path: Order.java
Layer: Architectural Compatibility Alias
Total Lines: 13 lines of Java source code

Functional Role: Root-level compatibility model extending com.kcau.dining.model.Order. Preserves root model inheritance and backward compatibility.

Complete Java Source Code:

```java
/**
 * Root-level Model for KCAU Smart Dining System
 * Extends the modular com.kcau.dining.model.Order
 */
public class Order extends com.kcau.dining.model.Order {
    public Order() {
        super();
    }

    public Order(String orderId, String userId, String customerName, double totalAmount, String status) {
        super(orderId, userId, customerName, totalAmount, status);
    }
}
```